



projeto 3

Prototipagem de Sistemas Computacionais - Experiência Prática 3



2 Identificação do aluno

SS

Samuel dos Santos Escoval CRUZ_EAD_Cst em Análise e
Desenvolvimento de Sistemas_1A_20261

projeto 3
Prototipagem de Sistemas
Computacionais - Experiência
Prática 3



3 Casos de uso

Casos de uso selecionados

- Analisar Viabilidade e Aprovar Proposta: Permite ao Proprietário avaliar os custos de uma licitação e decidir pela aprovação. O sistema calcula a margem de lucro e, se aprovado, transiciona o status da licitação para "Em Execução", gerando os registros financeiros necessários
- Definir Cardápio e Insumos: Gerencia a escolha do cardápio pelo Empregador, permitindo opções personalizadas. O sistema converte a escolha em uma lista técnica de insumos e adereços, vinculando os custos reais à proposta aprovada para controle de gastos.



4 Identificação de elementos

Nome da classe

- Proposta
- Licitação
- Cardápio
- EventoOperacional
- Logística

Atributos principais

- dProposta: int valorTotal: double margemLucro: double statusAprovacao: boolean
- numEdital: String orgaoEmpregador: String objeto: String statusProcesso: String
- idCardapio: int tipo: String listatens: String custoInsumos: double personalizado: boolean.
- idEvento: int dataEvento: Date local: String statusPreparo: String permanenciaEquipe: boolean
- idLogistica: int transporteProprio: boolean custoFrete: double statusEntrega: String

Métodos principais

- calcularViabilidade(): Valida se a margem de lucro é positiva após subtrair custos de insumos e frete; autorizarExecucao(): Altera o status para aprovado e dispara a ordem de serviço; gerarFaturamento(): Processa a cobrança final conforme a condição de pagamento acordada.
- vincularProposta(): Realiza a conexão lógica entre o edital externo e a proposta interna; atualizarStatus(): Sincroniza as etapas da licitação web com o banco de dados do sistema para garantir a rastreabilidade total.
- definirItens(): Gerencia a seleção de pratos padrão ou a inserção de itens personalizados; calcularCustoInsumos(): Realiza o cálculo automático do valor total de custo de todos os ingredientes e materiais necessários para a execução do buffet.
- iniciarMontagem(): Registra o início do preparo no local; apurarPerdas(): Contabiliza materiais danificados ou extraviados para ajuste no fechamento financeiro; registrarConclusao(): Finaliza formalmente a operação e libera os recursos no sistema
- alocarTransporte(): Define o uso de frota própria ou inicia a contratação de frete terceirizado; confirmarEntrega(): Registra o cumprimento do prazo logístico e a chegada do buffet ao destino final definido na licitação



5 Definição de relacionamentos

Classe de origem - Classe de destino

- Licitação - Proposta
- Proposta - Cardápio
- Proposta - Evento Operacional
- Evento Operacional - Logística
- Licitação - PropostaComercial
- PropostaComercial - Cardápio
- Proposta Comercial - Evento Operacional
- Evento Operacional - Logística

Tipo de relacionamento

- Associação
- Composição
- Associação
- Composição
- Associação
- Composição
- Associação
- Composição

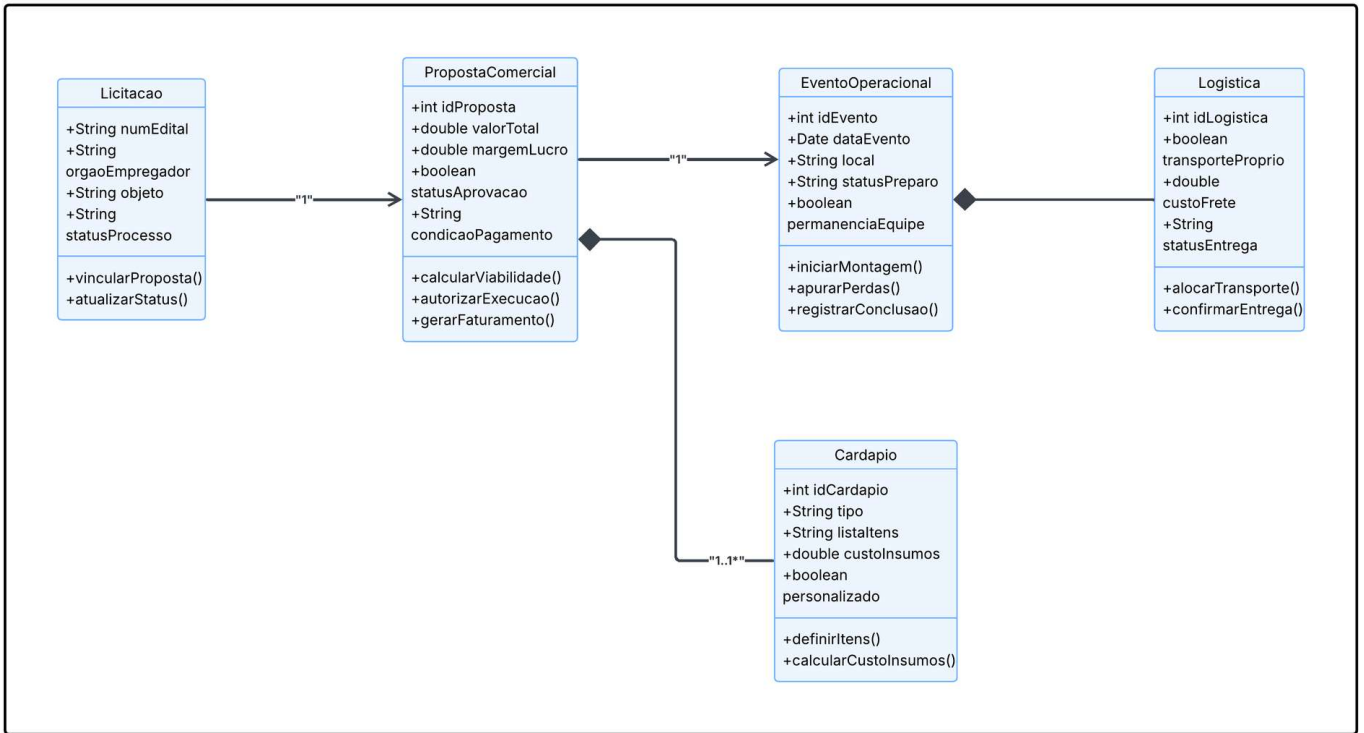
Multiplicidade

- 1:1
- 1 : 1..*
- 1:1
- 1:1
- 1:1
- 1 : 1..*
- 1:1
- 1 : 1



6 Diagrama de classes

SGPD Buffet – Diagrama de Classes UML





7 Mapeamento de interações

Caso de uso para diagrama de sequência

Analisar Viabilidade e Aprovar Proposta Escolhi este caso de uso por ser o ponto crítico de controle do sistema. Ele é o caso de uso que melhor valida a integração entre as classes construídas anteriormente (Licitação, PropostaComercial e Cardápio). É nesta interação que as regras de negócio financeiras (gastos vs. lucros) são processadas, determinando se o fluxo operacional deve ser disparado ou se o processo deve ser encerrado, minimizando riscos de prejuízo para o Proprietário.

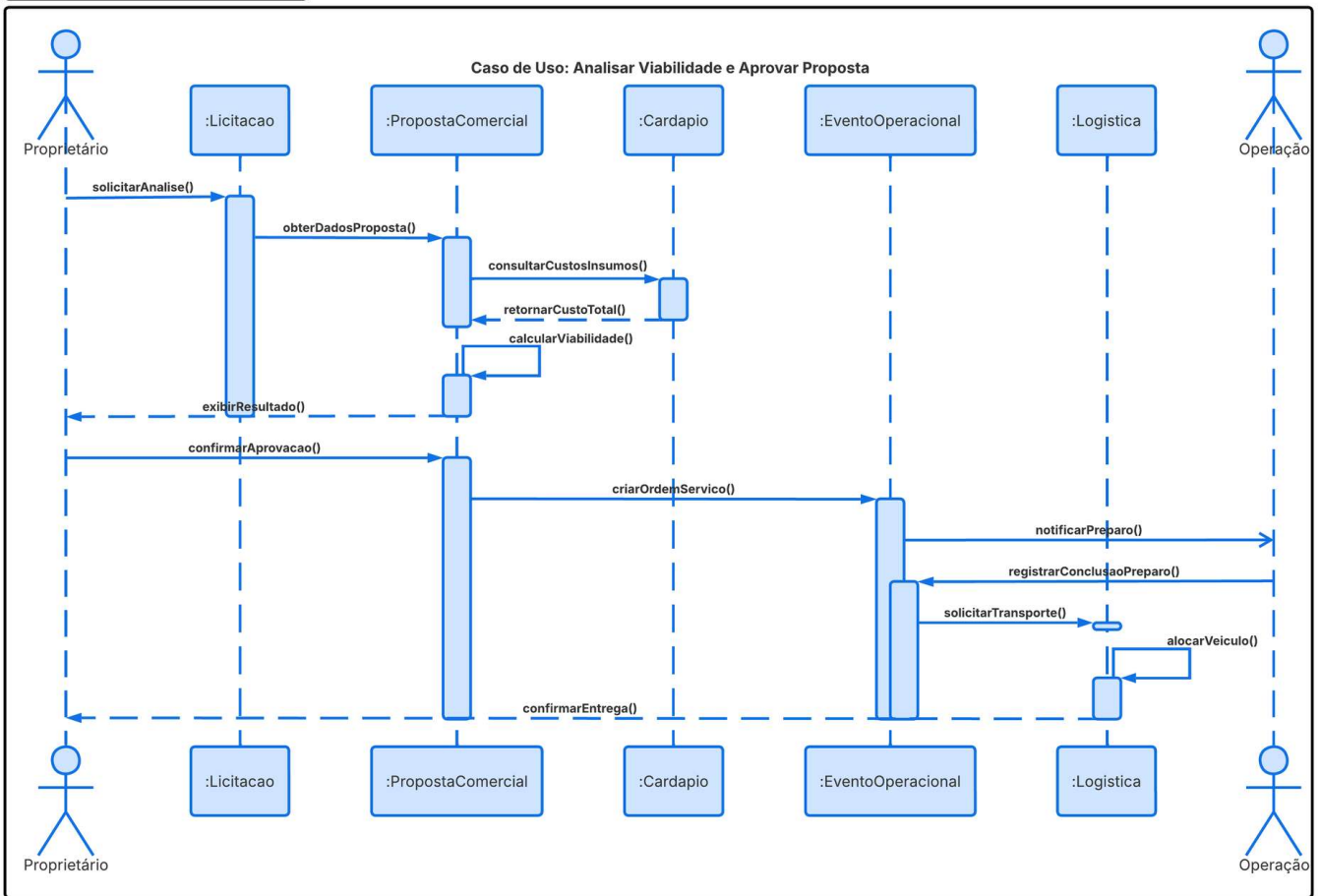
Lista de interações

- Proprietário (:Licitação): solicitarAnalise(idLicitação)
- :Licitação (:Proposta): obterDadosProposta()
- :Proposta (:Cardápio): consultarCustosInsumos()
- :Cardápio (:Proposta): retornarCustoTotal()
- :Proposta (:Proposta(self-call)): calcularViabilidade()
- :Proposta (Proprietário): exibirResultado()
- Proprietário (:Proposta): confirmarAprovacao()
- :Proposta (:EventoOperacional): criarOrdemServico()
- :EventoOperacional (Operação): notificarPreparo()
- Operação (:EventoOperacional): registrarConclusaoPreparo()
- :EventoOperacional (:Logística): solicitarTransporte()
- :Logística (:Logística(self-call)): alocarVeiculo()
- :Logística (Proprietário): confirmarEntrega()



8 Diagrama de sequência

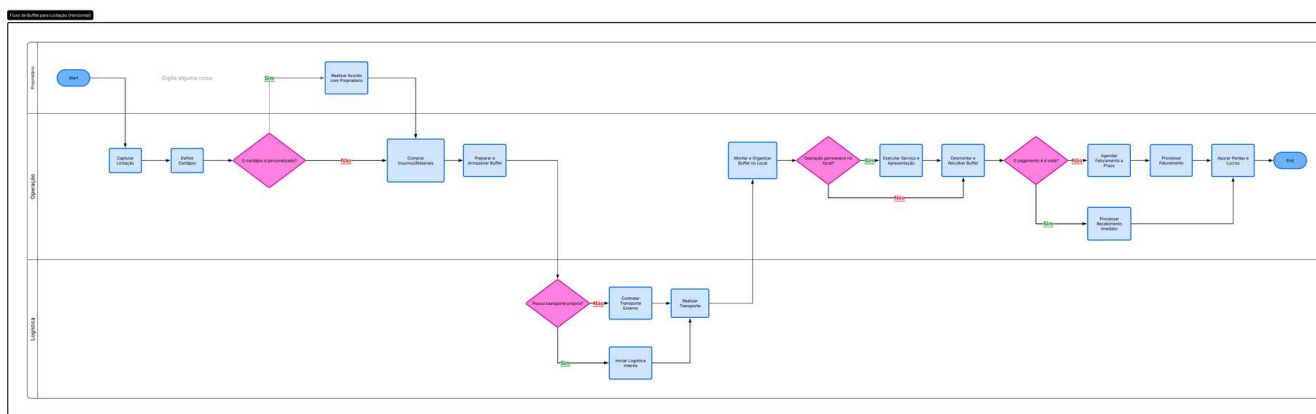
Caso de Uso: Analisar Viabilidade e Aprovar Proposta





Mapeamento de passos do fluxo

- Capturar Licitação: Definir Cardápio
- Definir Cardápio: [Decisão] O cardápio é personalizado?
- [Decisão] Personalizado?: Se Sim: Realizar Acordo com Proprietário / Se Não: Seguir para Compra de Insumos
- Comprar Insumos/Materiais: Preparar e Armazenar Buffet
- Preparar e Armazenar Buffet: [Decisão] Possui transporte próprio?
- [Decisão] Transporte Próprio?: Se Sim: Iniciar Logística Interna / Se Não: Contratar Transporte Externo
- Realizar Transporte: Montar e Organizar Buffet no Local
- Montar e Organizar Buffet: [Decisão] Operação permanece no local?
- [Decisão] Operação permanece no local?: Se Sim: Executar Serviço e Apresentação / Se Não: Desmontar e Recolher após evento
- Desmontar e Recolher Buffet: [Decisão] O pagamento é à vista?
- [Decisão] Pagamento à Vista?: Se Sim: Processar Recebimento Imediato / Se Não: Agendar Faturamento a Prazo
- Processar Faturamento: Apurar Perdas e Lucros
- Apurar Perdas e Lucros: Fim do Processo





Comentário técnico

Ao finalizar a modelagem do Sistema de Gestão de Processos Digitais (SGPD) para o setor de Buffet, fica claro que um diagrama não funciona sem o outro. Durante o desenvolvimento, percebi que o Diagrama de Classes é o "chão" onde o sistema pisa; ele é a base estrutural que guarda todas as informações. Mas são os diagramas de Sequência e de Atividades que mostram como essas informações "correm" dentro do software no dia a dia. A Base Estrutural ditando o Comportamento. No meu trabalho, a consistência aparece quando olhamos, por exemplo, para a classe Proposta. No Diagrama de Classes, eu defini que ela tem atributos como `valorTotal` e `margemLucro`. Sem esses dados "parados" lá, eu jamais conseguiria realizar a "conversa" que mostrei no Diagrama de Sequência. Lá, o objeto de Proposta só consegue responder ao Proprietário se o negócio é viável porque ele tem o método `calcularViabilidade()` e os atributos financeiros prontos para serem usados. Se eu não tivesse criado essa estrutura antes, o sistema "travaria" por falta de informação no momento da decisão. Validando a Operação com os Diagramas Comportamentais. O Diagrama de Atividades foi onde eu realmente consegui ver os erros e gargalos da operação de buffet. Por exemplo, ao mapear as decisões (os losangos rosa), como a escolha entre "Transporte Próprio" ou "Terceirizado", eu precisei voltar no meu Diagrama de Classes e garantir que a classe Logística tivesse o atributo `transporteProprio: boolean`. Isso prova que a modelagem comportamental serve como um teste: se eu desenhei um processo no fluxo de atividades, mas não tenho uma classe ou método para suportar aquilo, o sistema falharia na vida real. Visão Holística e Melhoria Contínua Essa integração me permitiu entender que o software não é apenas uma sequência de tarefas, mas uma colaboração entre objetos. Quando a Operação registra a conclusão do preparo, isso altera um status que a Logística vai ler para iniciar o transporte. Ver essa "engrenagem" funcionando antes mesmo de escrever qualquer linha de código é o que permite melhorar o processo. Se eu noto que o faturamento demora muito no fluxo de atividades, posso repensar os métodos da classe `PropostaComercial` para automatizar o repasse à operação e o cálculo de perdas de forma mais rápida. Em resumo, o Diagrama de Classes deu a inteligência e a memória ao sistema, enquanto os diagramas de Sequência e Atividades deram o movimento e a lógica. Essa visão completa garante que o SGPD seja um sistema robusto, capaz de evitar prejuízos e organizar a bagunça que pode ser a gestão de um buffet sem um controle digital rigoroso.



12 Revisão da experiência prática

Checklist de qualidade

- Revisão e seleção de ao menos dois casos de uso principais da etapa anterior foram realizadas com sucesso.
- Identificação de classes, atributos e métodos principais do sistema foi executada de forma estruturada.
- Definição de relacionamentos e multiplicidades foi aplicada utilizando boas práticas de design orientado a objetos.
- Elaboração do Diagrama de Sequência mapeou adequadamente a interação entre objetos ao longo do tempo.
- Construção do Diagrama de Atividades refletiu o fluxo lógico e o comportamento descrito no caso de uso.
- Produção do comentário técnico interpretou corretamente a complementaridade entre os modelos estruturais e comportamentais.



Reflexão sobre as aprendizagens

Nesta experiência prática eu consegui perceber o quanto a minha visão sobre sistemas mudou desde o primeiro diagrama que fiz. No começo, eu estava muito focado apenas no processo do buffet no mundo real (processo esse que acabei por esquecer de atualizar na primeira pratica, o que descarrou da segunda e terceira pratica, por eu, ainda não ter tido o insight que seria tudo uma atividade continua..)e tive bastante dificuldade para entender que no diagrama de atividades e de sequência o foco precisava ser o software e não apenas as pessoas. Um dos meus pontos fortes foi conseguir manter a lógica do negócio desde o início e não desistir de entender o rigor técnico da UML mesmo quando a mente já estava fritando de tanto detalhe. Eu vejo como oportunidade de melhoria a parte de diferenciar melhor os atores externos das funções internas do sistema já que no começo eu queria colocar raia para tudo e precisei de ajuda para entender que nem tudo vira uma raia ou um caso de uso. Outro ponto que sofri um pouco foi com a simbologia correta como saber quando usar um losango rosa para decisão ou como fazer as setas de auto-chamada e as mensagens de retorno de forma que o programador entendesse o fluxo. Esses aprendizados são fundamentais para minha carreira porque agora eu sei que desenhar um sistema antes de programar ajuda a ver erros de lógica como os problemas que eu tive com as setas soltas ou caminhos sem fim que poderiam travar o software na vida real. Ter essa visão completa de como as classes guardam os dados e como os diagramas de sequência fazem esses dados conversarem me dá muito mais segurança para trabalhar em projetos reais e entregar soluções que realmente funcionam e dão lucro.



14 Autoavaliação

